

Table: students (student_id INT, student_name VARCHAR(50), age INT, gender VARCHAR(10))

Basic SELECT Queries:

1. Get all student details

```
SELECT *  
FROM Students;
```

2. Get names of all students

```
SELECT Student_name  
FROM Students;
```

3. Get names and ages of students

```
SELECT Student_name, Age  
FROM Students;
```

4. Find students aged 20

```
SELECT Student_name, Age  
FROM Students,  
WHERE Age=20;
```

5. Find female students

```
SELECT Student_name  
FROM students  
WHERE Gender = 'Female';
```

Table: courses (course_id INT, course_name VARCHAR(100), teacher_name VARCHAR(50))

Table: marks (student_id INT, course_id INT, marks INT)

UPDATE, DELETE, INSERT

1. Update Manoj's age to 21

```
UPDATE Students  
SET Age=21  
WHERE student_name = ' Manoj'
```

2. Increase marks by 5 for all students

```
UPDATE students  
SET Marks = marks + 5;
```

3. Insert a new course

```
UPDATE students  
SET Course_name = 'Pharma'  
WHERE Student_ID = '5'
```

4. Add new student

```
INSERT INTO Students( Student_id,Student_name, Age, Gender, Course_ID, Course_name,  
Teacher_name, Marks)  
VALUE  
(31,Guru,21,Male,C131,BCA,Vijay,86)
```

Filtering and Conditions

1. Students older than 18

```
SELECT Student_name, Age  
FROM Students  
WHERE Age>18
```

2. Students aged between 18 and 22

```
SELECT Student_name, Age  
FROM Students,  
WHERE Age BETWEEN 18 AND 22;
```

3. Students whose names start with 'A'

```
SELECT * FROM students  
WHERE Student_name LIKE 'A%'
```

4. Students whose names contain 'an'

```
SELECT * FROM students
```

WHERE Student_name LIKE '%an%'

5. Students not aged 20

```
SELECT Student_name, Age
FROM Students
WHERE Age !=20
```

Or

```
SELECT * FROM Students
WHERE Age !=20
```

Or

```
SELECT Student_name, Age
FROM students
WHERE NOT Age = 20;
```

DISTINCT, ORDER BY, LIMIT

1. List all distinct ages

```
SELECT DISTINCT Age
FROM Students
```

Or

```
SELECT DISTINCT Age, Student_name
FROM Students
Group BY Age
```

2. Order students by age

```
SELECT Students_name, Age
FROM Students
ORDER BY Age DESC
```

3. Top 5 youngest students

```
SELECT Student_name, Age  
FROM Students  
ORDER BY Age ASC  
LIMIT 5;
```

4. Students ordered by name descending

```
SELECT * FROM Students,  
ORDER BY Student_name DESC
```

5. Oldest student

```
SELECT MAX(Age) AS Oldest_Student  
FROM students
```

Or

```
SELECT MAX(Age) AS Oldest_Student, Student_name  
FROM students
```

Aggregate Functions

1. Total number of students

```
SELECT COUNT(*)  
FROM students
```

2. Average age

```
SELECT AVG ( Age) AS AVG_AGE  
FROM students
```

Or

```
SELECT AVG ( Age) AS AVG_AGE, Student_name  
FROM students
```

3. Maximum age

```
SELECT MAX(Age)
FROM students
```

4. Number of male and female students

```
SELECT Gender, COUNT(*) AS Total_Students
FROM students
GROUP BY Gender;
```

5. Count students per age

```
SELECT Age, COUNT(*) As total_students
FROM students
GROUP BY Age
```

JOINS and Relationships

1. Show student name, course name, and marks

```
SELECT s.student_name,
c.course_name,
m.marks
FROM students s
INNER JOIN marks m
ON s.student_id = m.student_id
INNER JOIN courses c
ON m.course_id = c.course_id;
```

2. Show only students who have marks in any course

```
SELECT s.student_name
FROM students s
INNER JOIN marks m
ON s.student_id=m.student_id
WHERE m.marks IS NOT NULL;
```

3. Courses taught by 'Prof. Lakshmi '

```
SELECT *  
FROM Courses  
WHERE Teacher_name = 'Lakshmi';
```

4. Students who scored above 80 in any course

```
SELECT s.Student_name,  
c.course_name,  
m.marks  
FROM students s  
INNER JOIN marks m  
ON s.student_id=m.student_id  
INNER JOIN courses c  
ON m.course_id=c.course_id  
WHERE marks > 80
```

5. Students who did not appear in marks table

```
SELECT *  
FROM Students s  
LEFT JOIN Marks m  
ON s.student_id = m.student_id  
WHERE m.student_id IS NULL;
```

Group By + Having

1. Average marks per course

```
SELECT c.course_name,  
AVG(m.marks) AS Average_Marks  
FROM courses c  
INNER JOIN marks m  
ON c.course_id=m.course_id
```

```
GROUP BY c.course_name  
HAVING AVG(m.marks)>0;
```

2. Students with average marks > 70

```
SELECT  
    s.student_id,  
    s.student_name,  
    c.course_name,  
    AVG(m.marks) AS average_marks  
FROM students s  
INNER JOIN marks m  
    ON s.student_id = m.student_id  
INNER JOIN courses c  
    ON m.course_id = c.course_id  
GROUP BY  
    s.student_id,  
    s.student_name,  
    c.course_name  
HAVING AVG(m.marks) > 70;
```

3. Courses where max marks is above 90

```
SELECT  
    c.course_id,  
    c.course_name,  
    MAX(m.marks) AS max_marks  
FROM courses c  
INNER JOIN marks m  
    ON c.course_id = m.course_id  
INNER JOIN students s  
    ON s.student_id = m.student_id  
GROUP BY  
    c.course_id,  
    c.course_name  
HAVING MAX(m.marks) > 90;
```

4. Count students per course

```
SELECT
    c.course_id,
    c.course_name,
    COUNT(s.student_id) AS total_students
FROM courses c
LEFT JOIN marks m
    ON c.course_id = m.course_id
LEFT JOIN students s
    ON s.student_id = m.student_id
GROUP BY
    c.course_id,
    c.course_name;
```

5. Total marks per student

```
SELECT
    s.student_id,
    s.student_name,
    SUM(m.marks) AS total_marks
FROM students s
INNER JOIN marks m
    ON s.student_id = m.student_id
GROUP BY
    s.student_id,
    s.student_name;
```